Assignment 6: Smart Home

Observer and Visitor Patterns | Variant 7

Anuar Nurlybek | Software Design Patterns | 31 October 2026

Repository

https://github.com/Anuarrich0/Assignment-6

The repository includes the complete project, JUnit tests, README with design decisions, UML class diagram and two trade-off commits.

Observer

Two sources publish device energy samples and motion events. Five listeners are wired in Demo.main: MobileApp, EnergyTracker, SecurityAlarm, EventLog and InspectionService. Our generic Subject<E> and Observer<E> interfaces define the contract. HomeEvent is an immutable typed record.

Push notifications carry complete events, avoiding a later read from mutable source state. Listeners subscribe per EventType and may join or leave inside a handler. EnergyTracker also implements Subject: it accumulates Wh per room and republishes ENERGY_SUMMARY events.

Composite and Visitor

The immutable tree has three levels: Home - Room - device. Home and Room are containers. Original leaves are Lamp, Thermostat, Camera and DoorLock; a later commit adds SmokeDetector. Elements contain data, validation and accept only.

Visitor<R> uses double dispatch: runtime dispatch selects the concrete accept method, then visitor.visit(this) selects the overload for that concrete element type. Visitors own traversal so an operation can start at the home or one room.

PowerVisitor: numeric power use.
InventoryVisitor: stateful counts across the complete tree.
SecurityVisitor: list of unsafe devices.
JsonVisitor: nested JSON with escaped strings.
ReplacementCostVisitor: replacement budget using fixed demo prices.

Integration

Positive MOTION triggers InspectionService, which looks up the named Room, runs SecurityVisitor on that subtree only and publishes INSPECTION_REPORT. EventLog records the result. A test checks that another room is excluded. The device tree is an immutable inspection snapshot; energy events are incremental samples rather than live tree mutations.

Concurrency and validation

Thread safety and shutdown

ProducerRunner uses an ExecutorService with a configurable worker count, default three. The demo submits three producers. EventSource stores listeners in eagerly initialized CopyOnWriteArraySets per event type. Callbacks execute without a whole-subject lock and iterate a snapshot. Removing a listener affects later snapshots; an already captured callback may still run.

ConcurrentHashMap protects energy totals and mobile state; AtomicInteger protects alarm counters; ConcurrentLinkedQueue stores logs. A failed listener does not prevent other callbacks: failures are collected and then propagated. Concurrent publishers do not promise global ordering.

Publishing is synchronous. invokeAll, Future.get, shutdown and awaitTermination wait for normal accepted work and expose producer failures. External interruption requests cancellation and propagates interruption; lossless completion applies to normal shutdown. Visitors with mutable accumulated state are created per report.

Test results

mvn clean verify: 27 tests passed, 0 failures, 0 errors, 0 skipped. The compiler targets Java 17 with release 17; local execution used JDK 21. JUnit 5 is the only project dependency and is test-scoped.

The stress test synchronizes four workers, each publishing 1,000 events. Expected and observed: 4,000 source deliveries, 4,000 summary deliveries and 4,000 Wh. Further tests cover callback subscription changes, concurrent unsubscribe, listener and producer failures, immutable data, invalid inputs, JSON escaping/nesting, subtree isolation and the new leaf in every visitor.

Hand calculations: the test room consumes 10 + 100 + 0 + 1 = **111 W** and has two security findings. The executable demo matches **1014 W**, **300 Wh**, 100 alerts and **600 logged events**.

Trade-off proof

1349834 - new visitor: ReplacementCostVisitor.java, ReplacementCostTest.java and README.md only. No element changed. Fixture cost: 20 + 150 + 80 + 60 = 310 units. Adding an operation requires implementing one method for each existing element type.

406ea37 - new element: SmokeDetector.java, Visitor.java, all five visitors, SmokeDetectorTest.java and README.md. Existing containers and original leaves remain unchanged. A new element requires updating every visitor. The README lists exact paths. Dates reflect actual implementation time; no two-week development history is claimed.

Run

```
mvn clean verify
java -cp target/classes home.Demo
java -cp target/classes home.Demo 4
```